

Computer Architecture: A Quantitative Approach

Hennessy & Patterson

September 24, 2026

Contents

1	Fundamentals of Quantitative Design and Analysis	2
1.1	Introduction	2
1.2	Classes of Computers	2
1.3	Defining Computer Architecture	2
1.4	Trends in Technology	3
1.5	Trends in Power and Energy in Integrated Circuits	4
1.6	Trends in Cost	5
1.7	Dependability	5
1.8	Measuring, Reporting, and Summarizing Performance	5
1.9	Quantitative Principles of Computer Design	5
1.9.1	Amdahl's Law	6
A	Appendix A: Instruction Set Principles	7
A.1	Introduction	7
A.2	Classifying Instruction Set Architectures	7
A.3	Memory Addressing	7
A.4	Types and Sizes of Operands	8
A.5	Operations in the Instruction Set	8
A.6	Instructions for Control Flow	8
A.6.1	Addressing modes for control flow instructions	9
A.6.2	Conditional branch options	9
A.6.3	Procedure Invocation Options	10
A.7	Encoding an Instruction Set	10
A.8	Cross-Cutting Issues: The Role of Compilers	11
A.8.1	Register Allocation	11
B	Review of Memory Hierarchy	12
B.1	Introduction	12
B.1.1	Where can a block be placed in a cache?	13
B.1.2	How is a block found if it is in the cache?	13
B.1.3	Which block should be replaced on a cache miss?	13
B.1.4	What happens on a write?	14

Order: Ch1, Appendix A, Appendix B, Appendix C, Ch2, Ch5, Ch3, Ch4, Ch7, Ch6

1 Fundamentals of Quantitative Design and Analysis

1.1 Introduction

- RISC - Reduced Instruction Set Computer was introduced in the early 1980s, with the aim of simplifying architectures to keep up with the rapid development the hardware.
- Utilized instruction level parallelism (initially with pipelining) and the use of caches.
- ARM became dominant in the late 90s as a RISC architecture, specifically for low-end power and cost sensitive applications.

1.2 Classes of Computers

- There exist two main types of parallelism:
 1. Data level parallelism (DLP) when there are many data items that may be parallelized.
 2. Task level parallelism (TLP) when larger tasks can operate independent of each other and may be parallelized.
- Hardware can then exploit these types of parallelism in four ways:
 1. Instruction-level Parallelism (ILP) exploits DLP with the help of the compiler with ideas like pipelining and speculation.
 2. Vector Architectures and Graphic Processing Units (GPUs) exploit DLP by applying a single instruction to multiple pieces of data in parallel.
 3. Thread-Level Parallelism exploits either DLP or TLP in a specific hardware model that allows faster and efficient communication between threads
 4. Request-Level Parallelism exploits TLP as specified by the OS or programmer.
- Michael Flynn in the 60s all computers one of the following four categories sorted by parallelism in the instruction and data streams:
 1. *Single instruction, single data* (SISD) - A uniprocessor. Instructions are executed sequentially, but can still exploit ILP.
 2. *Single instruction, multiple data* (SIMD) - There are multiple processors with independent data streams executing the same instruction in parallel. Each processor has its own data memory but they share the same instruction memory.
 3. *Multiple instruction, single data* (MISD) - Doesn't really exist
 4. *Multiple instruction, multiple data* (MIMD) - Targets TLP as each processor can fetch their own instructions and data. Generally more flexible and applicable than SIMD, but is inherently more expensive.

1.3 Defining Computer Architecture

- Instruction set architecture (ISA): the set of instructions for the computer that are visible to the programmer. It serves as the boundary between software and hardware.
 1. Class of ISA: Nearly all modern ISAs operate on registers or memory locations. 80x86 has 16 general-purpose regs and 16 floating-point regs, whereas MIPS has 32 of each.
 - Register-memory ISA: Can access memory with a variety of instructions (eg 80x86)

- Load-store ISA: Can only access memory via load/store instructions (eg ARM and MIPS)
- 2. Memory addressing: Nearly all architectures use byte addressing to identify memory locations. Some architectures require an address to be aligned to a certain multiple (like ARM or MIPS) while others do not, but may have a performance cost.
- 3. Addressing modes:
 - MIPS: Register, Immediate (constant only), Displacement
 - 80x86: Same with three variants of displacement - no register (absolute), two registers (based indexed with displacement), and two registers (based with scaled index and displacement)
 - ARM: Same as MIPS plus PC-relative addressing, sum of two registers, and the sum of two registers where one register is multiplied by the size of the operand in bytes.
- 4. Types and sizes of operands: All 3 support operands sizes of 8-bit ascii, 16-bit unicode/half-word, 32-bit integer/word, 64-bit double word/long, 32-bit IEEE 754 floating point @ single precision, and 64-bit IEEE 754 floating point @ double precision. 80x86 also supports 80-bit extended floating point @ extended double precision.
- 5. Operations: Categories include data transfer, arithmetic logical, control, and floating point.
- 6. Control flow operations: Nearly all ISAs support conditional branching, unconditional jumps, procedure calls, and returns. Address is relative to program counter (PC).
- 7. Encoding an ISA: fixed length (eg ARM and MIPS, all instructions are 32 bits long) or variable length (eg 80x86, instructions are 1-18 bytes long). Variable length binaries tend to be smaller as instructions can be packed together more.
- Organization (or microarchitecture) refers to the high-level abstracted representation of a computer's design, such as the memory system, memory interconnect, and the design of the internal processor or CPU.
- Two processors may implement the same ISA but have different organizations (eg AMD Opteron vs Intel Core i7).
- The hardware is the lower level specifics of a computer.
- Two processors may implement the same ISA and organization but vary in physical hardware (eg Intel Core i7 vs Intel Xeon 7560).
- *Architecture* is an overarching term that refers to the instruction set architecture, organization/microarchitecture, and hardware.

1.4 Trends in Technology

- Well-designed ISAs are created with the intention to survive more rapid changes in technology.
 1. Integrated circuit logic technology: Transistor density (historically) has increased by 35% each year, approximately quadrupling over 4 years. Increases in die size range from 10%-20%, and the transistor count on a chip doubles every approx. 18-24 months (Moore's law)

2. Semiconductor DRAM: Capacity per DRAM chip has increased by 25%-40% each year. This is slowing down however due to the difficulty of efficiently manufacturing small DRAM cells.
 3. Semiconductor flash (electrically erasable programable read-only memory): Capacity per flash chip has increased 50%-60% per year recently, and is 15-20 times cheaper than DRAM.
 4. Magnetic disk technology: Density has increased 40% per year since 2004. 15-25 times cheaper than flash and 300-500 times cheaper than DRAM, per bit.
 5. Network technology: Depends both on the performance of switches and the performance of the transmission system (hard to estimate)
- Bandwidth or Throughput: total amount of work done in a given time
 - Latency or Response Time: Time between start and completion of a given task
 - Generally, bandwidth outpaces latency improvements; generally bandwidth grows by at least the square of latency.
 - Feature Size: minimum size of a transistor in either dimension.
 - Density of transistors increases quadratically wrt. a decrease in feature size
 - Generally, transistor performance increases linearly wrt. a decrease in feature size
 - As feature size shrinks, however, wire performance degrades as they get shorter and the resistance and capacitance per unit length gets worse.

1.5 Trends in Power and Energy in Integrated Circuits

- Three primary concerns from the perspective of a system designer:
 1. What is the maximum power a processor requires?
 2. What is the sustained power consumption? Quantified via thermal design power (TDP), which determines the cooling requirement. Peak power is approx. 1.5 times TDP
 3. Energy efficiency
- Dynamic energy: energy required to switch a transistor in CMOS chips.

$$\text{Energy}_{\text{dynamic}} \propto \text{Capacitive Load} \times \text{Voltage}^2$$

Represents the energy pulse of the transition of $0 \rightarrow 1 \rightarrow 0$ (or $1 \rightarrow 0 \rightarrow 1$). Thus the energy of a single transition is then

$$\begin{aligned} \text{Energy}_{\text{dynamic}} &\propto \frac{1}{2} \times \text{Capacitive Load} \times \text{Voltage}^2, \\ \text{Power}_{\text{dynamic}} &\propto \frac{1}{2} \times \text{Capacitive Load} \times \text{Voltage}^2 \times \text{Frequency Switched} \end{aligned}$$

- As technology evolves the increase of the number of transistors and frequency makes the decrease of load capacitance and voltage negligible, creating an overall increase of power consumption.
- Early microprocessors used < 1 watt; modern processors consume on the order of 100s of watts.

- Power consumption has largely plateaued, as the issue of cooling the chips and preventing hot spots becomes increasingly more difficult.
- Various techniques exist to try and improve efficiency:
 1. Do nothing: Turn off the clock on certain inactive modules
 2. Dynamic Voltage-Frequency Scaling (DVFS): When not operating at full capacity the clock rate will be lowered.
 3. Overclocking: Newer processors may allow certain cores to be able to operate at higher clock speeds for short periods of time, before having to cool off.

1.6 Trends in Cost

- While the cost of manufacturing an IC has decreased dramatically, the overall process has not changed: a wafer gets tested and cut into dies.
- Die yield: the fraction of good dies on a wafer relative to total dies. Empirically,

$$\text{Die Yield} = \text{Wafer Yield} \times \frac{1}{(1 + \text{Defects per Unit Area} \times \text{Die Area})^N}$$

- Wafer yield refers to the number of dies that are so bad they need not be tested; for simplicity this can be assumed to be ~100%.
- In 2010, Defects per unit area was reported to be 0.016-0.057 defects per square centimeter.
- N is called the process-complexity factor, a measure of manufacturing difficulty. for 40 nm in 2010, this ranged from 11.5-15.5.
- To accomodate inevitable defects in manufacturing, many designers introduce redundant cells in the dies such that small defects still yield a functional die. This significantly increases the yield.

1.7 Dependability

- Historically, ICs have been one of, if not the most reliable part of a computer.
- While pins may be fragile the overall error rate inside the chip is very low.
- Service Level Agreement/Service Level Objective (SLA/SLO): A guarantee given by an infrastructure provider that their service would be dependable.
- Two states: Service Accomplishment (delivered as specified) or Interruption (different from SLA).

1.8 Measuring, Reporting, and Summarizing Performance

1.9 Quantitative Principles of Computer Design

- Take advantage of parallelism: various techniques exist at different scales where tasks and/or data can be parallelized.
 - System level: Consider a web service; the handling of multiple requests can be spread out across multiple processors to improve throughput.
 - Single processor: Pipelining is when the processor will overlap one instruction while an earlier one is being executed to reduce the wait time between them. The key

insight is that not every instruction depends on the previous one, so executing instructions partially or fully in parallel may be possible.

- Digital design: Modern ALUs use carry-lookahead, which use parallelism to speed the process of computing sums from linear to logarithmic to the number of bits.
- Principle of Locality: Keep things that are used often close.
 - As a rule of thumb, a program will spend 90% of its runtime on just 10% of the code.
 - Based off of this we can predict what data and instructions a program will use based off of what the program has already done.
 - Temporal locality: recently accessed items are more likely to be accessed again in the future.
 - Spatial locality: items with similar addresses tend to be referenced close together in time.

1.9.1 Amdahl's Law

Amdahl's Law defines the speedup that can be gained by a particular feature.

$$\text{Speedup} = \frac{\text{Performance for entire task using enhancement when possible}}{\text{Performance for entire task without using the enhancement}}$$

As well,

$$\text{Execution Time}_{\text{new}} = \text{Execution Time}_{\text{old}} \times ((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}})$$

Where $\text{Fraction}_{\text{enhanced}}$ is the fraction of the program that benefits from the enhancement, and $\text{Speedup}_{\text{enhanced}}$ represents how much faster said portion of the program is. Thus, the overall speedup ratio is then

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution Time}_{\text{old}}}{\text{Execution Time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

A Appendix A: Instruction Set Principles

A.1 Introduction

A.2 Classifying Instruction Set Architectures

- Main types of internal storage:
 1. Stack: Operands are implicitly on top of the stack
 2. Accumulator: One operand is implicitly on top of the accumulator
 3. General-purpose register: Only explicit operands
- Consider the expression $C = A + B$:

Stack	Accumulator	Register-memory	Register (load-store)
Push A	Load B	Load R1,A	Load R1,A
Push B	Add B	Add R1,B	Load R2,B
Add	Store C	Store R1,C	Add R3,R1,R2
Pop C			Store R3,C

- Types of GPR architectures:
- Register-register (load/store): arithmetic operations can only operate on registers; memory is fetched through load/store operations. Takes 0-3 operands. Very fast and low CPI. (eg RISC)
- Register-memory: arithmetic operations can have one register and one memory operand. Takes 0-2 operands. Higher CPI, but code density is much higher than register-register.
- Memory-memory: arithmetic operations can fetch both operands from memory. Much slower and pipelining is nearly impossible as memory ops can stall. Essentially extinct in modern day architectures

A.3 Memory Addressing

- Big endian vs little endian: Consider the number 0x12345678. If we were to store this in address 0x00:

Memory Address	Big Endian	Little Endian
0x00	12 (MSB)	78 (LSB)
0x01	34	56
0x02	56	34
0x03	78	12

- Note that endianness does not affect the order of the bits themselves, but the bytes.
- Big endian is easier to read (follows left $\rightarrow$ right), but little endian is a little easier for the processor as the ALU processes bytes from LSB $\rightarrow$ MSB. Sign checking is also easier on little endian.
- CPUs will fetch memory in units known as words. If an object is in a misaligned memory address (an access to a object of size s bytes with byte address A where $A \bmod s \neq 0$) will require multiple aligned memory accesses.
- Addressing mode: How architectures specify the address of an object they will access. The address specified by the mode is called the effective address.

Addressing Mode	Example Instruction	Meaning	When Used
Register	Add R3,R4	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register
Immediate	Add R4,#3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	When a value is a constant
Displacement	Add R4,100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables
Register indirect	Add R4,(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Access through a pointer
Indexed	Add R3,(R1+R2)	$\text{Regs}[R3] \leftarrow \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Array indexing
Direct/absolute	Add R1,(1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Static data access

A.4 Types and Sizes of Operands

- Generally, the type of an operand (integer, single-precision floating point, char, etc) determines the size of the operand.
- Integers are almost always represented as twos-compliment binary numbers.
- Twos compliment: negative numbers are represented by inverting all bits and adding 1 (MSB becomes sign bit)
- Some architectures may also represent the operands as decimal values, since certain decimal fractions (like 0.1) do not have a binary representation.

A.5 Operations in the Instruction Set

Operator Type	Examples
Arithmetic and logical	Integer operations: add, subtract, and, or, multiply, divide, etc.
Data transfer	Loads and stores (mov instructions on architectures with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	syscall, virtual memory management instructions
Floating-point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, multiply, decimal->char conversion
String	String move, cmp, search
Graphics	Pixel and vertex, compression/decompression operations

- The first 4 (Arithmetic, and logical, data transfer, control, and system) are the important ones
- The top 10 most common instructions are responsible for 96% of all instructions executed (on 80x86):
- 22% Load, 20% Conditional branch, 16% Compare, 12% Store

A.6 Instructions for Control Flow

- 4 types of control flow change:
 1. Conditional branch
 2. Jump

- 3. Procedure call
- 4. Procedure return
- Majority is Conditional branch (~80%)

A.6.1 Addressing modes for control flow instructions

- The destination address must always be specified.
- In the majority of cases, this is specified explicitly in the instruction
- Procedure return is the main exception, as the return address is not known at compile time (just pop call stack)
- Most common mode is via displacement relative to the program counter (PC).
- Instructions that address this way are known as pc-relative instructions
- Advantages are that these instructions take less bytes as the location is close to the current instruction.
- Position independence: PC-relative instructions also allow code to be run independent of the absolute location.
- Position independence helps with linking/dynamic linking.
- When PC-relative addressing is not available, the next best option is usually register indirect.
- This is due to the fact that PC-relative is not possible when the address is not known at compile time.
- Register-indirect jumps also allow the following:
 1. Switch-case functions
 2. Runtime dynamic dispatch
 3. Function pointers
 4. Dynamically shared libraries

A.6.2 Conditional branch options

Below are ways in which conditional branches are evaluated:

1. Condition code (CC): Branch operations will read specific flags set by the ALU (as a side effect) and branch accordingly. This has the advantage that it may be free as the arithmetic being done may have set the flag already. The disadvantage is that this depends on the state of the ALU flags, and can easily be overwritten restricting reordering of instructions. As well, the outcome is passed through the flags register to the branch, which is implicit (more dependencies for the compiler to consider) and there is usually only one flag register slot. Many modern architectures like x86 intel/amd, aarch64, and powerPC use CC.
2. Condition register/limited comparison: A compare instruction will write a result into any GPR, which is then checked against a predicate. This doesn't rely on any flag states and is much simpler for the compiler to reason about. The disadvantage is that some conditions (eg $a < b$) introduce an extra compare instruction before the branching itself. This introduces a design tension, where either the branch's compare can be trivial (which

introduces extra instructions) OR the branch's compare can be more powerful (but affect the critical path).

3. Compare and branch: The compare operation is folded into the branch instruction itself, thus no extra instructions/flags are required, at the cost of a more expensive instruction.

A.6.3 Procedure Invocation Options

On procedure calls some notion of state saving is needed; for example the return address. Newer architectures require the compiler to explicitly generate loads/stores for each register that is to be saved/restored.

- Caller saving: the calling procedure is responsible for saving registers that it wants preserved after returning
- Callee saving: the called procedure is responsible.

A mix of both is used to ensure that no values are lost during procedure calls.

```
1 void clobber_stuff(); // some function whos internals we dont control
2
3 int compute(int a, int b) {
4     int x = a + b;
5     clobber_stuff();
6     clobber_stuff();
7     clobber_stuff();
8     int y = x * 2; // we need to ensure x stays
9     return y;
10 }
```

Suppose x is in a caller-saving register. Since we dont know what clobber_stuff does, the compiler is obligated to move the contents somewhere else to preserve the value of x.

On the other hand, if x is in a callee-saving register, then the compiler doesn't need to do this, as it is the obligation of clobber_stuff to do preserve the value, if it is necessary.

Caller saving registers are also known as volatile, as the value held within the register may change without notice. On x86, an example is the rax register (rbx is callee saved).

A.7 Encoding an Instruction Set

- The choices for how instructions are encoded affects both the size of the binary and the speed of execution (processor must decode as quickly as possible).
- the Opcode is one field of the encoded instruction that specifies the specific instruction to be executed; it is paired with some number (possibly 0) of operands.
- The important decision is how the addressing mode is to be encoded.
- One must balance:
 1. Trying to have as many registers and addressing modes as possible
 2. The impact of having larger and more registers/addressing modes on the average instruction size and thus the average program size
 3. Having an instruction length that the processor can figure out nearly instantly, for the sake of pipelining

- For variable length encodings, the processor does not know when instruction N+1 starts until it has finished instruction N, which hurts the fetch/decode stages of pipelining.

Operation and no. of Operands	Address specifier 1	Address field 1	...	Address specifier n	Address field n
----------------------------------	------------------------	--------------------	-----	--------------------------	----------------------

Table 1: Variable

Operation	Address field 1	Address field 2	Address field 3
-----------	--------------------	--------------------	--------------------

Table 2: Fixed

Operation	Address specifier	Address field	
Operation	Address specifier 1	Address specifier 2	Address field
Operation	Address specifier 1	Address field 1	Address field 2

Table 3: Hybrid (opcode specifies length)

For example, in x86 (variable):

`add EAX,1000(EBX)`

- 32-bit integer add instruction with two operands; the opcode takes 1 byte.
- The address specifier is a ModRM byte, which has 3 subfields:
 - Addressing mode class (MOD): 2 bits
 - Register operand (Reg): 3 bits
 - Base register (combined with MOD to get the raddressing mode, rm): 3 bits
- Note that there may also be an extra byte with the ModRM byte called the SIB (scale-index-base) byte
- the displacement, which is 1 or 4 bytes; Since we are displacing by 1000, we need to use 4 bytes to represent this (for small displacements from -128 to +127, it is done with one byte).
- Overall this instruction takes 6 bytes. However, if the displacement was smaller, it could take a minimum of 3.

A.8 Cross-Cutting Issues: The Role of Compilers

Phase-ordering problem: Certain compiler optimizations are too expensive to undo, and thus they must predetermine the order in which optimization passes are done. For example, inlining procedure calls have to be done before knowing the exact size of the procedure being called.

A.8.1 Register Allocation

- Due to the importance of proper register allocation, it is one of the most important optimizations the compiler performs.
- Graph coloring algorithms are used, where a graph is constructed that represents all possible candidates for allocation to a register.
- Graph coloring however is an NP-complete problem, so heuristics are used

B Review of Memory Hierarchy

B.1 Introduction

- *Cache*: The name given to the first level of memory which is encountered once an address leaves the processor.
 - Caches are also applied whenever buffering is applied to store frequently or recently used items, such as file caches, name caches, etc.
- *Cache hit*: When the processor finds the requested piece of data in the cache; if it is not in a cache it is a *cache miss*.
- *Block*: A fixed size collection of data that is retrieved when accessing a requested word not currently in the cache.
- *Temporal locality*: How likely this word is going to be used in the near future (thus it should be placed in a cache so it can be accessed quickly).
- *Spacial locality*: How likely data near the word accessed is going to be used (thus data in similar locations should be fetched together)
- *Latency*: How long it takes to fetch the first word of a block (time to retrieve)
- *Bandwidth*: How long it takes to fetch the rest of the block (throughput)
- Cache misses are handled by hardware and forces processors to use in-order execution processors to stall, however out-of-order processors may be able to continue with instructions that don't use that piece of data.
- *Page*: A fixed-size block of address space; at a given time each page resides either in main memory or on disk.
- *Page fault*: When a processor references an item not in a cache nor in a page residing in main memory. Handled in software so that the processor does not stall.
- *Memory stall cycles*: The number of cycles the processor is stalled waiting for a memory access.

$$\text{CPU Execution time} = (\text{CPU Clock Cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

- *Miss penalty*: The cost of a single miss, i.e. the extra cycles spent servicing it

$$\begin{aligned}\text{Miss Penalty} &= \text{Access latency} + \frac{\text{Block Size}}{\text{Bandwidth}} \\ &\approx \frac{\text{Total Memory Stall Cycles}}{\text{Number of Misses}}\end{aligned}$$

$$\text{Memory Stall Cycles} = \text{Number of misses} \times \text{Miss penalty}$$

$$\begin{aligned}&= \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \\ &= \text{IC} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss Penalty}\end{aligned}$$

- Note that "Memory Access" includes instruction fetches as well; a program that is 50% memory accesses will have 1.0 accesses for fetching the instruction itself (always happens) + 0.5 accesses for loads/stores = 1.5 memory accesses per instruction.
- The last form is the easiest to measure on hardware. The average miss penalty via the stall cycles and number of misses is empirical and generally used.
- Miss rates can be measured with cache simulators, which take an address trace (a log of all memory addressing operations done) to simulate the cache behaviour and determine the number of total misses and hits.
- Many modern microprocessors have builtin counters that can measure miss rate as well.

B.1.1 Where can a block be placed in a cache?

- If there is a one-to-one mapping between placements and blocks, then the cache is *directly mapped*, with a mapping

$$A \bmod n_B \tag{1}$$

- Where A is the block address and n_B is the number of blocks in the cache.
- If a block can be placed anywhere, then the cache is *fully associative*.
- If a block can be placed in a restricted set of places in the cache, then the cache is *set associative*, where a set is a group of blocks in the cache.
- A block is first mapped to a set (similar to direct mapping) via

$$A \bmod n_S \tag{2}$$

- Where n_S is the number of sets, and the block is placed anywhere in the set.
- The cache placement is then called *n-way set associative*, where n is the number of blocks in a given set.
- Most L1 caches are 8-way and L2/L3 caches often 8- to 16-way.
- Ways and sets are orthogonal to each other

B.1.2 How is a block found if it is in the cache?

- Each block frame has an address tag that gives the block an address.
- Ideally all tags are searched in parallel.
- A *valid bit* is used to say whether or not the entry contains a valid address.
- A full address is divided into 4 sections:
 1. Block Address: Can be further divided into a tag field and index field.
 2. The index field describes which set the block is in
 3. The tag field specifies which block in the set may contain the information.
 4. The block offset selects the desired data from the block.
- The tag field must be searched in parallel since there does not exist a bijective mapping from addresses to block frames (there are too many addresses for not enough blocks).
- While the entire address may be searched, the offset bit would be trivial since the entire block is either present or not (a block cannot be half loaded), and the index is redundant as it was used to select which set the block is in.
- Increasing associativity increases the number of blocks per set and decreases the size of the index which increases the size of the tag. Fully associative caches have no index, and thus have to check every address in parallel.

B.1.3 Which block should be replaced on a cache miss?

When a miss occurs, a block must be selected to be evicted to make space for a different block.

- Random - Candidate blocks are randomly selected to ensure a uniform allocation spread.
- Least recently used (LRU) - Accesses to blocks are recorded, and the block that has been unused for the longest is evicted. This reduces the chances of throwing out information that may be needed soon, based off of the principle that recently used blocks are likely to be used again.
- First in, First out (FIFO) - An alternative to LRU as real LRU is expensive to calculate; FIFO can approximate LRU by determining the oldest block instead of the LRU.

Real processors usually implement a pseudo-LRU; When a way in a given set is accessed, turn on a bit. When all bits in a set are on, turn off all bits except for the most recently accessed item. When a block needs to be evicted, choose randomly from any bit that is off

B.1.4 What happens on a write?

- Since reads are much more common than writes the cache is optimized for reads; processors also need to wait for reads to finish but not necessarily for writes.
- As well, the block addresses are in a separate structure than the actual data itself (split in a tag array and data array). Once the processor indexes into the correct set, the data (at the block offset) in all ways of the set are fetched; there is a n:1 mux at the end which returns the block associated with the correct tag.
- These are done at the same time; if the read is a hit, then the requested part is passed to the processor immediately, and if it is a miss there is no harm to performance (the values read are ignored).
- This cannot be done for writes, as modifying a block cannot be done until the exact block to modify has been found.
- As well, more than enough bytes can be returned on a read; with writes, the exact number of bytes specified are allowed to be modified.
- Two write policies:
 1. *Write through*: Information is written into the block in the cache as well as the block in lower-level memory.
 2. *Write back*: Information is only written into the block in the cache; the block is then only written to the next level down when evicted.
- For write-back caches, they also often have a *dirty bit*, which signifies whether that block has been modified; if it is clean, then there is no need to write the block back into the next level down (since it has not been changed).
- Write-back is better for latency as multiple writes within a block require only one write to the lower level memory.
- Write through is simpler to implement as the cache block is always clean; write through also guarantees that the next level down will always contain the most recent copy of the data (easier for coherency).
- *Write stall*: When the processor must wait for writes to complete.
- *Write buffer*: Write instructions are queued in a buffer, overlapping processor execution with memory updating (write stalls are still possible!)
- Since the processor doesn't need the data after a write, there are two options on a write miss:
 1. *Write allocate*: On write miss, allocate the block and proceed with the write hit actions. Write misses act exactly like read misses.
 2. *No-write allocate*: On write miss, modify the block in-place in the lower-level memory

Assuming fully associative write-back that starts empty:

```
1 Write Mem[100]; // 1
2 Write Mem[100]; // 2
3 Read  Mem[200]; // 3
4 Write Mem[200]; // 4
5 Write Mem[100]; // 5
```

For no-write allocate:

1. Miss
2. Miss (Mem[100] was not allocated)
3. Miss
4. Hit
5. Miss

For write-allocate:

1. Miss
2. Hit (Mem[100] was allocated)

3. Miss
4. Hit
5. Hit